

ЛИСТИНГ ПРОГРАММЫ

Листинг 1 – Класс «Движение учеников» (DvizhenieUcheniakovEntity)

```

public class DvizhenieUcheniakovEntity
{
    // Первичный ключ
    [Key]
    public long DvizhenieID { get; set; }
    // Внешний ключ, ссылка на ученика
    [Required(ErrorMessage =
"Идентификатор ученика обязателен")]
    public long UchenikID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("UchenikID")]
    public virtual UchenikEntity Uchenik
    { get; set; }
    // Дата изменения
    [Required(ErrorMessage = "Дата
изменения обязательна")]

    public DateTime DataIzmeneniya { get;
set; }
    // Тип движения (например, перевод,
отчисление, восстановление)
    [Required(ErrorMessage = "Тип
движения обязателен"), StringLength(50)]
    public string TipDvizheniya { get;
set; }
    // Основание (причина) движения
    [Required(ErrorMessage = "Основание
обязательно"), StringLength(255)]
    public string Osnovanie { get; set; }
    // Дополнительные комментарии
    [StringLength(500)] // Допустимо
    больше текста
    public string Kommentariy { get; set;
}
}

```

Листинг 2 – Класс «Финансы» (FinansyEntity)

```

public class FinansyEntity
{
    // Первичный ключ
    [Key]
    public long PlatezhID { get; set; }
    // Внешний ключ, ссылка на учащийся
    [Required(ErrorMessage =
"Идентификатор учащегося обязателен")]
    public long UchenikID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("UchenikID")]
    public virtual UchenikEntity Uchenik
    { get; set; }
    // Дата платежа
    [Required(ErrorMessage = "Дата
платежа обязательна")]
    public DateTime DataPlatezha { get;
set; }

    // Сумма платежа
    [Required(ErrorMessage = "Сумма
платежа обязательна")]
    [Column(TypeName = "DECIMAL(18,2)")]
    public decimal Summa { get; set; }
    // Назначение платежа
    [Required(ErrorMessage = "Назначение
платежа обязательно"), StringLength(255)]
    public string Naznachenie { get; set;
}
    // Тип операции
    [Required(ErrorMessage = "Тип
операции обязателен"), StringLength(50)]
    public string TipOperacii { get; set;
}
}

```

Листинг 3 – Класс «Класс» (KlassEntity)

```
public class KlassEntity
{
    // Первичный ключ
    [Key]
    public long KlassID { get; set; }
    // Номер класса (например, 5А, 9В)
    [Required(ErrorMessage = "Номер
класса обязателен"), StringLength(10)]
    public string NomerKlassa { get; set; }
}

    // Учебный год
    [Required(ErrorMessage = "Учебный год
обязателен")]
    public int GodObucheniya { get; set; }
}

    // Внешний ключ, ссылка на учителя-
руководителя класса
    [Required(ErrorMessage =
"Руководитель класса обязателен")]

    public long KlassRukovoditelID { get;
set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("KlassRukovoditelID")]
    public virtual SotrudnikEntity
Sotrudnik { get; set; }
    // Внешний ключ, ссылка на учебный
план класса
    [Required(ErrorMessage = "План
обязателен")]
    public long PlanID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("PlanID")]
    public virtual UchebniyPlanEntity
UchebniyPlan { get; set; }
}
```

Листинг 4 – Класс «Материально-техническая база» (MatTehBazaEntity)

```
public class MatTehBazaEntity
{
    // Первичный ключ
    [Key]
    public int InventarID { get; set; }
    // Наименование имущества
    [Required(ErrorMessage =
"Наименование обязательно"),
StringLength(255)]
    public string Naimenovanie { get;
set; }
    // Тип имущества (оборудование,
мебель и т.д.)
    [Required(ErrorMessage = "Тип
обязателен"), StringLength(50)]
    public string Tip { get; set; }
    // Кабинет размещения
    [Required(ErrorMessage = "Кабинет
обязателен"), StringLength(10)]
    public string Kabinet { get; set; }
    // Инвентарный номер
    [Required(ErrorMessage = "Инвентарный
номер обязателен"), StringLength(20)]
    public string InvNomer { get; set; }
    // Статус имущества (в наличии,
списано и т.д.)

    [Required(ErrorMessage = "Статус
обязателен"), StringLength(50)]
    public string Status { get; set; }
    // Внешний ключ, ссылка на приказ о
постановке на учет
    [Required(ErrorMessage = "Приказ
постановки обязателен")]
    public int PrikazPostupleniyaID {
get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("PrikazPostupleniyaID")]
    public virtual PrikazEntity
PrikazPostupleniya { get; set; }
    // Внешний ключ, ссылка на приказ о
списании
    public int? PrikazSpisaniyaID { get;
set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("PrikazSpisaniyaID")]
    public virtual PrikazEntity
PrikazSpisaniya { get; set; }
}
```

Листинг 5 – Класс «Приказ» (PriказEntity)

```
public class PriказEntity
{
    // Первичный ключ
    [Key]
    public int PriказID { get; set; }
    // Номер приказа
    [Required(ErrorMessage = "Номер
приказа обязателен"), StringLength(50)]
    public string NomerPriказа { get;
set; }
    // Дата приказа
    [Required(ErrorMessage = "Дата
приказа обязательна")]
    public DateTime DataPriказа { get;
set; }
    // Тип приказа (например, приказ о
зачислении, переводе и т.д.)
    [Required(ErrorMessage = "Тип приказа
обязателен"), StringLength(100)]
    public string TipPriказа { get; set;
}

    // Содержание приказа
    [Required(ErrorMessage = "Содержание
обязательно"), Column(TypeName = "TEXT")]
    public string Soderzhanie { get; set;
}
    // Ссылка на файл документа
    [StringLength(255)]
    public string FileLink { get; set; }
    // Внешний ключ, ссылка на
сотрудника, издавшего приказ
    [Required(ErrorMessage = "Ссылка на
сотрудника обязательна")]
    public long SotrudnikID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("SotrudnikID")]
    public virtual SotrudnikEntity
Sotrudnik { get; set; }
}
```

Листинг 6 – Класс «Расписание» (RaspisanieEntity)

```
public class RaspisanieEntity
{
    // Первичный ключ
    [Key]
    public long RaspisanieID { get; set;
}
    // Внешний ключ, ссылка на класс
    [Required(ErrorMessage = "Класс
обязателен")]
    public long KlassID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("KlassID")]
    public virtual KlassEntity Klass {
get; set; }
    // Внешний ключ, ссылка на предмет
    [Required(ErrorMessage = "Предмет
обязателен")]
    public long PredmetID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("PredmetID")]
    public virtual UchebniyPredmetEntity
UchebniyPredmet { get; set; }

    // Внешний ключ, ссылка на
преподавателя
    [Required(ErrorMessage =
"Преподаватель обязателен")]
    public long SotrudnikID { get; set; }
    /// <summary> Навигационное свойство.
</summary>
    [ForeignKey("SotrudnikID")]
    public virtual SotrudnikEntity
Sotrudnik { get; set; }
    // День недели (1-понедельник, ...,
7-воскресенье)
    [Required(ErrorMessage = "День недели
обязателен")]
    public long DenNedeli { get; set; }
    // Номер урока в расписании
    [Required(ErrorMessage = "Номер урока
обязателен")]
    public short NomerUroka { get; set; }
    // Кабинет проведения занятия
    [Required(ErrorMessage = "Кабинет
обязателен"), StringLength(10)]
    public string Kabinet { get; set; }
}
```

Листинг 7 – Класс «Сообщения» (SoobschenieEntity)

```
public class SoobschenieEntity
{
    // Первичный ключ
    [Key]
    public int SoobschenieID { get; set; }
}

// Внешний ключ, отправитель
сообщения
[Required(ErrorMessage = "Отправитель
обязателен")]
public long OtpravitelID { get; set; }

/// <summary> Навигационное свойство.
</summary>
[ForeignKey("OtpravitelID")]
public virtual SotrudnikEntity
Otpravitel { get; set; }
// Внешний ключ, получатель сообщения
[Required(ErrorMessage = "Получатель
обязателен")]
public long PoluchatelID { get; set; }

/// <summary> Навигационное свойство.
</summary>
[ForeignKey("PoluchatelID")]

    public virtual SotrudnikEntity
    Poluchatel { get; set; }
    // Тип получателя (ученик, учитель,
    родитель и т.д.)
    [Required(ErrorMessage = "Тип
    получателя обязателен"), StringLength(50)]
    public string TipPoluchatelya { get;
    set; }
    // Тема сообщения
    [Required(ErrorMessage = "Тема
    обязательна"), StringLength(255)]
    public string Tema { get; set; }
    // Текст сообщения
    [Required(ErrorMessage = "Текст
    обязателен"), Column(TypeName = "TEXT")]
    public string Text { get; set; }
    // Дата отправки
    [Required(ErrorMessage = "Дата
    отправки обязательна")]
    public DateTime DataOtpravki { get;
    set; }
    // Прочитано (булево поле, true -
    прочитано, false - непрочитано)
    public bool Prochitano { get; set; }
}
```

Листинг 8 – Класс «Сотрудники» (SotrudnikEntity)

```
public class SotrudnikEntity
{
    // Первичный ключ
    [Key]
    public long SotrudnikID { get; set; }
    // Фамилия
    [Required(ErrorMessage = "Фамилия
    обязательна"), StringLength(100)]
    public string Familia { get; set; }
    // Имя
    [Required(ErrorMessage = "Имя
    обязательно"), StringLength(100)]
    public string Imya { get; set; }
    // Отчество
    [StringLength(100)]
    public string Otchestvo { get; set; }
    // Дата рождения
    [Required(ErrorMessage = "Дата
    рождения обязательна")]
    public DateTime DataRozhdeniya { get;
    set; }

    // Должность
    [Required(ErrorMessage = "Должность
    обязательна"), StringLength(100)]
    public string Dolzhnost { get; set; }
    // Email
    [StringLength(255)]
    public string Email { get; set; }
    // Телефон
    [StringLength(20)]
    public string Telefon { get; set; }
    // Дата приема
    [Required(ErrorMessage = "Дата приема
    обязательна")]
    public DateTime DataPriema { get;
    set; }
    // Статус
    [StringLength(50)]
    public string Status { get; set; }
}
```

Листинг 9 – Класс «Учебный план» (UchebniyPlanEntity)

```
public class UchebniyPlanEntity
{
    [Key]
    public long PlanID { get; set; }
    [Required(ErrorMessage = "Название обязательно"), StringLength(255)]
    public string Nazvanie { get; set; }
    // Год начала учебного плана
    [Required(ErrorMessage = "Год обязателен")]
    public int GodNachala { get; set; }
    // Описание учебного плана
    [Column(TypeName = "TEXT")] // Для больших объемов текста
    public string Opisanie { get; set; }
}
```

Листинг 10 – Класс «Учебный предмет» (UchebniyPredmetEntity)

```
public class UchebniyPredmetEntity
{
    // Первичный ключ
    [Key]
    public long PredmetID { get; set; }
    // Название предмета
    [Required(ErrorMessage = "Название
обязательно"), StringLength(255)]
    public string Nazvanie { get; set; }
    // Сокращение (аббревиатура)
    [Required(ErrorMessage = "Сокращение
обязательно"), StringLength(10)]
    public string Sokrashenie { get; set; }
    // Количество часов в неделю
    [Required(ErrorMessage = "Количество
часов обязательно")]
    public int ChasovNedelyu { get; set; }
}
```

Листинг 11 – Класс «Ученик» (UchenikEntity)

```
public class UchenikEntity
{
    // Первичный ключ
    [Key]
    public long UchenikID { get; set; }
    // Фамилия
    [Required(ErrorMessage = "Фамилия
обязательна"), StringLength(100)]
    public string Familiia { get; set; }
    // Имя
    [Required(ErrorMessage = "Имя
обязательно"), StringLength(100)]
    public string Imya { get; set; }
    // Отчество
    [StringLength(100)]
    public string Otchestvo { get; set; }
    // Дата рождения
    [Required(ErrorMessage = "Дата
рождения обязательна")]
    public DateTime DataRozhdeniya { get;
set; }
    // Адрес проживания
    [Required(ErrorMessage = "Адрес
проживания обязателен"), StringLength(255)]
    public string AdresProzhivaniya {
get; set; }
    // ФИО родителей
    [Required(ErrorMessage = "ФИО
родителей обязательно"), StringLength(255)]
    public string FIORoditeley { get;
set; }
    // Телефон родителя
    [StringLength(20)]
    public string TelefonRoditelya { get;
set; }
    // Дата зачисления
    [Required(ErrorMessage = "Дата
зачисления обязательна")]
    public DateTime DataZachisleniya {
get; set; }
    // Класс ID
    [Required(ErrorMessage = "Класс
обязателен")]
    public long KlassID { get; set; }
    /// <summary> Навигационное свойство
для роли. </summary>
    [ForeignKey("KlassID")]
    public virtual KlassEntity Klass {
get; set; }
}
```

Листинг 12 – Класс «Журнал успеваемости» (ZhurnalUspevaemostiEntity)

```
public class ZhurnalUspevaemostiEntity
{
    // Первичный ключ
    [Key]
    public long ZapisID { get; set; }
    // Внешний ключ, ссылка на расписание
занятий
    [Required(ErrorMessage = "Расписание
обязательно")]
    public long RaspisanieID { get; set; }
}
/// <summary> Навигационное свойство.
</summary>
[ForeignKey("RaspisanieID")]
public virtual RaspisanieEntity
Raspisanie { get; set; }
// Внешний ключ, ссылка на ученика
[Required(ErrorMessage = "Ученик
обязателен")]
public long UchenikID { get; set; }
/// <summary> Навигационное свойство.
</summary>
[ForeignKey("UchenikID")]
public virtual UchenikEntity Uchenik
{ get; set; }
[Required(ErrorMessage = "Дата урока
обязательна")]
public DateTime DataUroka { get; set; }
// Оценка (числовое значение оценки)
public int Osenka { get; set; }
// Тип оценки (контрольная работа,
самостоятельная работа и т.д.)
[Required(ErrorMessage = "Тип оценки
обязателен"), StringLength(50)]
public string TipOcenki { get; set; }
// Посещаемость (true -
присутствовал, false - отсутствовал)
public bool Poseschaemost { get; set; }
// Причина отсутствия (если
отсутствует)
[StringLength(255)]
public string PrichinaOtssutstviya {
get; set; }
}
```

Листинг 13 – Реализация сущности Role в подсистеме аутентификации

```
namespace [MaxLength(50)]
SchoolAssistancePlatform.Base.Entity.Auth;
// Класс роли
[Table("Roles", Schema = "Auth")]
public class RoleEntity
{
    /// <summary> Идентификатор роли.
    </summary>
    [Key]
    public long Id { get; set; }

    /// <summary> Название роли.
    </summary>
    [Required]
    public string Name { get; set; }

    /// <summary> Описание роли.
    </summary>
    [MaxLength(255)]
    public string Description { get; set; }

    public virtual
    ICollection<RolePermissionEntity> Permissions
    { get; set; } = [];
}
```

Листинг 14 – Реализация связующей сущности RolePermission в подсистеме авторизации

```
namespace
SchoolAssistancePlatform.Base.Entity.Auth;
[Table("RolePermissions", Schema = "Auth")]
public class RolePermissionEntity
{
    [Key]
    public long Id { get; set; }

    public long RoleId { get; set; }
    public long PermissionValue { get; set; }

    [ForeignKey("RoleId")]
    public virtual RoleEntity Role { get; set; }
}
```

Листинг 15 – Реализация сущности User в подсистеме аутентификации (логин, хеш пароля, внешние ключи)

```
namespace
SchoolAssistancePlatform.Base.Entity.Auth;
// Класс пользователя
[Table("Users", Schema = "Auth")]
public class UserEntity
{
    /// <summary> Идентификатор
    пользователя. </summary>
    [Key]
    public long Id { get; set; }
    /// <summary> Логин пользователя.
    </summary>
    [Required]
    [MaxLength(50)]
    public string Login { get; set; }
    /// <summary> Хеш пароля. </summary>
    [Required]
    [MaxLength(255)]
    public string PasswordHash { get; set; }

    /// <summary> Электронная почта.
    </summary>
    [MaxLength(100)]
    public string Email { get; set; }

    /// <summary> Внешний ключ к
    сотруднику. </summary>
    public long? SotrudnikId { get; set; }

    /// <summary> Навигационное свойство.
    </summary>
    [ForeignKey("SotrudnikId")]
    public virtual SotrudnikEntity?
    Sotrudnik { get; set; }

    /// <summary> Внешний ключ к роли.
    </summary>
    public long RoleId { get; set; }
    /// <summary> Навигационное свойство.
    </summary>
    [ForeignKey("RoleId")]
    public virtual RoleEntity Role { get; set; }
}
```

Листинг 16 – Реализация контракта репозитория DvizhenieUcheniakov с методами фильтрации по ученику, типу и диапазону дат

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface
IDvizhenieUcheniakovRepository
{
    Task<DvizhenieUcheniakovDto>
CreateAsync(DvizhenieUcheniakovDto dto);
    Task<DvizhenieUcheniakovDto>
GetByIdAsync(long id);
    Task<IEnumerable<DvizhenieUcheniakovD
to>> GetAllAsync();
    Task<IEnumerable<DvizhenieUcheniakovD
to>> GetByUchenikAsync(long uchenikID);

    Task<IEnumerable<DvizhenieUcheniakovD
to>> GetByTipDvizheniyaAsync(string
tipDvizheniya);
    Task<IEnumerable<DvizhenieUcheniakovD
to>> GetByDateRangeAsync(DateTime startDate,
DateTime endDate);
    Task<DvizhenieUcheniakovDto>
UpdateAsync(long id, DvizhenieUcheniakovD
to);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
}
```

Листинг 17- Реализация контракта репозитория Finansy с методами получения баланса и суммы по ученику

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IFinansyRepository
{
    Task<FinansyDto>
CreateAsync(FinansyDto dto);
    Task<FinansyDto> GetByIdAsync(long
id);
    Task<IEnumerable<FinansyDto>>
GetAllAsync();
    Task<IEnumerable<FinansyDto>>
GetByUchenikAsync(long uchenikID);
    Task<IEnumerable<FinansyDto>>
GetByTipOperaciiAsync(string tipOperacii);

    Task<IEnumerable<FinansyDto>>
GetByDateRangeAsync(DateTime startDate,
DateTime endDate);
    Task<decimal>
GetTotalSumByUchenikAsync(long uchenikID);
    Task<decimal>
GetBalanceByUchenikAsync(long uchenikID);
    Task<FinansyDto> UpdateAsync(long id,
FinansyDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
}
```

Листинг 18 – Реализация контракта репозитория Klass с методами поиска по году обучения, классному руководителю и учебному плану

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IKlassRepository
{
    Task<KlassDto> CreateAsync(KlassDto
dto);
    Task<KlassDto> GetByIdAsync(long id);
    Task<IEnumerable<KlassDto>>
GetAllAsync();
    Task<IEnumerable<KlassDto>>
GetByGodObucheniyaAsync(int godObucheniya);

    Task<IEnumerable<KlassDto>>
GetByKlassRukovoditelAsync(long
klassRukovoditelID);
    Task<IEnumerable<KlassDto>>
GetByPlanAsync(long planID);
    Task<KlassDto> UpdateAsync(long id,
KlassDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
    Task<bool>
IsDuplicateKlassAsync(string nomerKlassa, int
godObucheniya, long? excludeId = null);
}
```

Листинг 19 – Реализация контракта репозитория MatTehBaza с методами поиска по типу, кабинету, статусу и приказу поступления

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IMatTehBazaRepository
{
    Task<MatTehBazaDto>
CreateAsync(MatTehBazaDto dto);
    Task<MatTehBazaDto> GetByIdAsync(int
id);
    Task<IEnumerable<MatTehBazaDto>>
GetAllAsync();
    Task<IEnumerable<MatTehBazaDto>>
GetByTipAsync(string tip);
    Task<IEnumerable<MatTehBazaDto>>
GetByKabinetAsync(string kabinet);

    Task<IEnumerable<MatTehBazaDto>>
GetByStatusAsync(string status);
    Task<IEnumerable<MatTehBazaDto>>
GetByPrikazPostupleniyaAsync(int
prikazPostupleniyaID);
    Task<MatTehBazaDto> UpdateAsync(int
id, MatTehBazaDto dto);
    Task<bool> DeleteAsync(int id);
    Task<bool> ExistsAsync(int id);
    Task<bool>
IsDuplicateInvNomerAsync(string invNomer,
int? excludeId = null);
}
```

Листинг 20 – Реализация контракта репозитория Prikaz с методами поиска по типу приказа, сотруднику и диапазону дат

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IPrikazRepository
{
    Task<PrikazDto> CreateAsync(PrikazDto
dto);
    Task<PrikazDto> GetByIdAsync(int id);
    Task<IEnumerable<PrikazDto>>
GetAllAsync();
    Task<IEnumerable<PrikazDto>>
GetByTipPrikazaAsync(string tipPrikaza);
    Task<IEnumerable<PrikazDto>>
GetBySotrudnikAsync(long sotrudnikID);

    Task<IEnumerable<PrikazDto>>
GetByDateRangeAsync(DateTime startDate,
DateTime endDate);
    Task<PrikazDto> UpdateAsync(int id,
PrikazDto dto);
    Task<bool> DeleteAsync(int id);
    Task<bool> ExistsAsync(int id);
    Task<bool>
IsDuplicateNomerPrikazaAsync(string
nomerPrikaza, int? excludeId = null);
}
```

Листинг 21 – Реализация контракта репозитория Raspisanie с методами поиска по классу, учителю, дню недели и проверкой конфликтов

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IRaspisanieRepository
{
    Task<RaspisanieDto>
CreateAsync(RaspisanieDto dto);
    Task<RaspisanieDto> GetByIdAsync(long
id);
    Task<IEnumerable<RaspisanieDto>>
GetAllAsync();
    Task<IEnumerable<RaspisanieDto>>
GetByKlassAsync(long klassID);
    Task<IEnumerable<RaspisanieDto>>
GetBySotrudnikAsync(long sotrudnikID);

    Task<IEnumerable<RaspisanieDto>>
GetByDenNedeliAsync(long denNedeli);
    Task<IEnumerable<RaspisanieDto>>
GetByKlassAndDenAsync(long klassID, long
denNedeli);
    Task<RaspisanieDto> UpdateAsync(long
id, RaspisanieDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
    Task<bool>
IsDuplicateScheduleAsync(long klassID, long
denNedeli, short nomerUroka, long? excludeId
= null);
}
```


Листинг 22 – Реализация контракта репозитория Soobschenie с методами поиска по отправителю, получателю, типу получателя и работе с непрочитанными сообщениями

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface ISoobschenieRepository
{
    Task<SoobschenieDto>
    CreateAsync(SoobschenieDto dto);
    Task<SoobschenieDto> GetByIdAsync(int
id);
    Task<IEnumerable<SoobschenieDto>>
    GetAllAsync();
    Task<IEnumerable<SoobschenieDto>>
    GetByOtpravitelAsync(long otpravitelID);
    Task<IEnumerable<SoobschenieDto>>
    GetByPoluchatelAsync(long poluchatelID);

    Task<IEnumerable<SoobschenieDto>>
    GetUnreadByPoluchatelAsync(long
poluchatelID);
    Task<IEnumerable<SoobschenieDto>>
    GetByTipPoluchatelyaAsync(string
tipPoluchatelya);
    Task<SoobschenieDto> UpdateAsync(int
id, SoobschenieDto dto);
    Task<bool> DeleteAsync(int id);
    Task<bool> ExistsAsync(int id);
    Task MarkAsReadAsync(int id);
    Task<int> GetUnreadCountAsync(long
poluchatelID);
}
```

Листинг 23 – Реализация контракта репозитория Sotrudnik с методами поиска по должности и получения активных сотрудников

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface ISotrudnikRepository
{
    Task<SotrudnikDto>
    CreateAsync(SotrudnikDto dto);
    Task<SotrudnikDto> GetByIdAsync(long
id);
    Task<IEnumerable<SotrudnikDto>>
    GetAllAsync();

    Task<IEnumerable<SotrudnikDto>>
    GetByDolzhnostAsync(string dolzhnost);
    Task<IEnumerable<SotrudnikDto>>
    GetActiveAsync();
    Task<SotrudnikDto> UpdateAsync(long
id, SotrudnikDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
}
```

Листинг 24 – Реализация контракта репозитория UchebniyPlan с методами поиска по году начала и названию

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IUchebniyPlanRepository
{
    Task<UchebniyPlanDto>
    CreateAsync(UchebniyPlanDto dto);
    Task<UchebniyPlanDto>
    GetByIdAsync(long id);
    Task<IEnumerable<UchebniyPlanDto>>
    GetAllAsync();

    Task<IEnumerable<UchebniyPlanDto>>
    GetByGodNachalaAsync(int godNachala);
    Task<IEnumerable<UchebniyPlanDto>>
    SearchByNazvanieAsync(string searchTerm);
    Task<UchebniyPlanDto>
    UpdateAsync(long id, UchebniyPlanDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
}
```

Листинг 25 – Реализация контракта репозитория UchebniyPredmet с методом проверки уникальности сокращения предмета

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IUchebniyPredmetRepository
{
    Task<UchebniyPredmetDto>
CreateAsync(UchebniyPredmetDto dto);
    Task<UchebniyPredmetDto>
GetByIdAsync(long id);
    Task<IEnumerable<UchebniyPredmetDto>>
GetAllAsync();

    Task<UchebniyPredmetDto>
UpdateAsync(long id, UchebniyPredmetDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
    Task<bool>
IsDuplicateSokrashenieAsync(string
sokrashenie, long? excludeId = null);
}
```

Листинг 26 – Реализация контракта репозитория Uchenik с методами поиска по классу, году обучения и фамилии

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface IUchenikRepository
{
    Task<UchenikDto>
CreateAsync(UchenikDto dto);
    Task<UchenikDto> GetByIdAsync(long
id);
    Task<IEnumerable<UchenikDto>>
GetAllAsync();

    Task<IEnumerable<UchenikDto>>
GetByKlassAsync(long klassID);
    Task<IEnumerable<UchenikDto>>
GetByGodObucheniyaAsync(int godObucheniya);
    Task<IEnumerable<UchenikDto>>
SearchByFamiliiaAsync(string familiia);
    Task<UchenikDto> UpdateAsync(long id,
UchenikDto dto);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
}
```

Листинг 27 – Реализация контракта репозитория ZhurnalUspevaemosti с методами поиска по ученику, расписанию, классу, диапазону дат и расчётом средней оценки

```
namespace
SchoolAssistancePlatform.Base.Interfaces;
public interface
IZhurnalUspevaemostiRepository
{
    Task<ZhurnalUspevaemostiDto>
CreateAsync(ZhurnalUspevaemostiDto dto);
    Task<ZhurnalUspevaemostiDto>
GetByIdAsync(long id);
    Task<IEnumerable<ZhurnalUspevaemostiD
to>> GetAllAsync();
    Task<IEnumerable<ZhurnalUspevaemostiD
to>> GetByUchenikAsync(long uchenikID);
    Task<IEnumerable<ZhurnalUspevaemostiD
to>> GetByRaspisanieAsync(long raspisanieID);

    Task<IEnumerable<ZhurnalUspevaemostiD
to>> GetByKlassAsync(long klassID);
    Task<IEnumerable<ZhurnalUspevaemostiD
to>> GetByDateRangeAsync(DateTime startDate,
DateTime endDate);
    Task<double>
GetAverageOcenkaByUchenikAsync(long
uchenikID);
    Task<ZhurnalUspevaemostiDto>
UpdateAsync(long id, ZhurnalUspevaemostiD
to);
    Task<bool> DeleteAsync(long id);
    Task<bool> ExistsAsync(long id);
}
```

Листинг 28 – Реализация DTO DvizhenieUcheniakov с дополнительными полями FIOUchenika и NomerKlassa»

```
namespace SchoolAssistancePlatform.framework.Data;
public class DvizhenieUcheniakovDto
{
    public long DvizhenieID { get; set; }
    public long UchenikID { get; set; }
    public string FIOUchenika { get; set; }
    public string NomerKlassa { get; set; }
    public DateTime DataIzmeneniya { get; set; }
    public string TipDvizheniya { get; set; }
    public string Osnovanie { get; set; }
    public string Kommentariy { get; set; }
}
```

Листинг 29 – Реализация DTO Finansy с дополнительными полями FIOUchenika и NomerKlassa

```
namespace SchoolAssistancePlatform.framework.Data;
public class FinansyDto
{
    public long PlatezhID { get; set; }
    public long UchenikID { get; set; }
    public string FIOUchenika { get; set; }
    public string NomerKlassa { get; set; }
    public DateTime DataPlatezha { get; set; }
    public decimal Summa { get; set; }
    public string Naznachenie { get; set; }
    public string TipOperacii { get; set; }
}
```

Листинг 30 – Реализация DTO Klass с дополнительными полями KlassRukovoditelFIO и PlanNazvanie

```
namespace SchoolAssistancePlatform.framework.Data;
public class KlassDto
{
    public long KlassID { get; set; }
    public string NomerKlassa { get; set; }
    public int GodObucheniya { get; set; }
    public long KlassRukovoditelID { get; set; }
    public string KlassRukovoditelFIO { get; set; }
    public long PlanID { get; set; }
    public string PlanNazvanie { get; set; }
}
```

Листинг 31 – Реализация DTO MatTehBaza с дополнительными полями номеров приказов поступления и списания

```
namespace SchoolAssistancePlatform.framework.Data;
public class MatTehBazaDto
{
    public int InventarID { get; set; }
    public string Naimenovanie { get; set; }
    public string Tip { get; set; }
    public string Kabinet { get; set; }
    public string InvNomer { get; set; }
    public string Status { get; set; }
    public int PrikazPostupleniyaID { get; set; }
    public string NomerPrikazaPostupleniya { get; set; }
    public int? PrikazSpisaniyaID { get; set; }
    public string NomerPrikazaSpisaniya { get; set; }
}
```

Листинг 32 – Реализация DTO Prikaz с дополнительным полем SotrudnikFIO namespace SchoolAssistancePlatform.framework.Data;

```
public class PrikazDto
{
    public int PrikazID { get; set; }
    public string NomerPrikaza { get; set; }
    public DateTime DataPrikaza { get; set; }
    public string TipPrikaza { get; set; }
    public string Soderzhanie { get; set; }
    public string FileLink { get; set; }
    public long SotrudnikID { get; set; }
    public string SotrudnikFIO { get; set; }
}
```

Листинг 33 – Реализация DTO Raspisanie с дополнительными полями NomerKlassa, NazvaniePredmeta, SokrasheniePredmeta, FIOPrepodatelya

```
namespace SchoolAssistancePlatform.framework.Data;
public class RaspisanieDto
{
    public long RaspisanieID { get; set; }
    public long KlassID { get; set; }
    public string NomerKlassa { get; set; }
    public long PredmetID { get; set; }
    public string NazvaniePredmeta { get; set; }
    public string SokrasheniePredmeta { get; set; }
    public long SotrudnikID { get; set; }
    public string FIOPrepodatelya { get; set; }
    public long DenNedeli { get; set; }
    public short NomerUroka { get; set; }
    public string Kabinet { get; set; }
}
```

Листинг 34 – Реализация DTO Role со списком разрешений Permissions

```
namespace SchoolAssistancePlatform.framework.Data;
public class RoleDto
{
    public long Id { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    public List<Permissions> Permissions { get; set; } = [];
}
```

Листинг 35 – Реализация DTO Soobschenie с дополнительными полями OtpravitelFIO и PoluchatelFIO

```
namespace SchoolAssistancePlatform.framework.Data;
public class SoobschenieDto
{
    public int SoobschenieID { get; set; }
    public long OtpravitelID { get; set; }
    public string OtpravitelFIO { get; set; }
    public long PoluchatelID { get; set; }
    public string PoluchatelFIO { get; set; }
    public string TipPoluchatelya { get; set; }
    public string Tema { get; set; }
    public string Text { get; set; }
    public DateTime DataOtpravki { get; set; }
    public bool Prochitano { get; set; }
}
```

Листинг 36 – Реализация DTO Sotrudnik со всеми основными полями кадрового учёта

```
namespace SchoolAssistancePlatform.framework.Data;
public class SotrudnikDto
{
    public long SotrudnikID { get; set; }
    public string Familia { get; set; }
    public string Imya { get; set; }
    public string Otchestvo { get; set; }
    public DateTime DataRozhdeniya { get; set; }
    public string Dolzhnost { get; set; }
    public string Email { get; set; }
    public string Telefon { get; set; }
    public DateTime DataPriema { get; set; }
    public string Status { get; set; }
}
```

Листинг 37 – Реализация DTO UchebniyPlan со всеми основными полями образовательной программы

```
namespace SchoolAssistancePlatform.framework.Data;
public class UchebniyPlanDto
{
    public long PlanID { get; set; }
    public string Nazvanie { get; set; }
    public int GodNachala { get; set; }
    public string Opisaniye { get; set; }
}
```

Листинг 38 – Реализация DTO UchebnyPredmet со всеми основными полями справочника дисциплин

```
namespace SchoolAssistancePlatform.framework.Data;
public class UchebnyPredmetDto
{
    public long PredmetID { get; set; }
    public string Nazvanie { get; set; }
    public string Sokrashenie { get; set; }
    public int ChasovNedelyu { get; set; }
}
```

Листинг 39 – Реализация DTO Uchenik с дополнительными полями NomerKlassa и GodObucheniya

```
namespace
SchoolAssistancePlatform.framework.Data;
public class UchenikDto
{
    public long UchenikID { get; set; }
    public string Familiia { get; set; }
    public string Imya { get; set; }
    public string Otchestvo { get; set; }
    public DateTime DataRozhdeniya { get;
set; }
    public string AdresProzhivaniya {
get; set; }
    public string FIORoditeley { get;
set; }
    public string TelefonRoditelya { get;
set; }
    public DateTime DataZachisleniya {
get; set; }
    public long KlassID { get; set; }
    public string NomerKlassa { get; set; }
    public int GodObucheniya { get; set; }
}
```

Листинг 40 – Реализация DTO User с адаптацией к Entity через Mapster и вложенным RoleDto

```
namespace SchoolAssistancePlatform.framework.Data;
[AdaptTo("[name]Entity")]
public class UserDto
{
    public long Id { get; set; }
    public string Login { get; set; }
    public string Email { get; set; }
    [AdaptMember("Role")]
    public RoleDto Role { get; set; }
}
```

Листинг 41 – Реализация DTO ZhurnalUspevaemosti с дополнительными полями NomerKlassa, NazvaniePredmeta, SokrasheniePredmeta, FIOUchenika

```
namespace
SchoolAssistancePlatform.framework.Data;
public class ZhurnalUspevaemostiDto
{
    public long ZapisID { get; set; }
    public long RaspisanieID { get; set; }
    public string NomerKlassa { get; set; }
    public string NazvaniePredmeta { get;
set; }
    public string SokrasheniePredmeta {
get; set; }
    public long UchenikID { get; set; }
    public string FIOUchenika { get; set; }
    public DateTime DataUroka { get; set; }
    public int Ocenka { get; set; }
    public string TipOcenki { get; set; }
    public bool Poseschaemost { get; set; }
    public string PrichinaOtssutstviya {
get; set; }
}
```

Листинг 42 – Реализация механизма навигации через событийную модель

```
namespace SchoolAssistancePlatform.UI.Actions;
internal class MenuActions
{
    public event EventHandler<string>? ChangeMenu;
    public void SelectMenuByName(string name)
    {
        ChangeMenu?.Invoke(this, name);
    }
}
```

Листинг 43 – Реализация DTO StudentGrade с коллекцией оценок, средним баллом и флагом редактирования

```
namespace SchoolAssistancePlatform.UI.Data;
public class StudentGradeDto
{
    public long StudentID { get; set; }
    public string StudentName { get; set; }
    public List<int> Grades { get; set; }
    public double AverageGrade { get; set; }
    public bool IsEditing { get; set; }
}
```

Листинг 44 – ViewModel авторизации с валидацией логина/пароля через AccountService и ReactiveUI

```
namespace
SchoolAssistancePlatform.UI.Views.Auth;
internal partial class AuthViewModel :
ReactiveObject
{
    private AccountService
_accountService;
    private string _login;
    private string _password;
    public string Login
    {
        get => _login;
        set =>
this.RaiseAndSetIfChanged(ref _login, value);
    }
    public string Password
    {
        get => _password;
        set =>
this.RaiseAndSetIfChanged(ref _password,
value);
    }
    public ReactiveCommand<Unit, Unit>
LoginCommand { get; }

    public AuthViewModel(AccountService
accountService)
    {
        _accountService =
accountService;
        LoginCommand =
ReactiveCommand.CreateFromTask(OnLoginAsync);
#if DEBUG
        _login = "admin";
        _password = "admin";
#endif
    }
    private async Task OnLoginAsync()
    {
        var correct = await
_accountService.CheckUserAsync(_login,
_password);
        if(correct)
        {
            await
_accountService.ChangeAccountAsync(_login,
_password);
            Login =
string.Empty;
            Password =
string.Empty;
        }
    }
}
```